검색 갈래 ③ 검색 엔진에 쓰기

문서 하나를 통째로 지우고 다시 넣어, 고친 문서의 옛 청크가 남지 않게 함



청크 단위로 교체하는 방식 (chunk_id 기준)

chunk_id로 덮어쓴 뒤 인덱스에 남은 청크

옛1 | 1′ | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10

11개 남음
옛 1번과 빠진 10번(고아)이 검색에 섞여 나옴

문서 단위로 교체하는 방식 (doc_id 기준)

doc_id가 같은 청크 10개 지움 → **새 청크 9개 넣음**

1′ | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

9개만 남음
고친 내용만 검색되고 고아 청크 없음

검색 엔진에 넣는 한 건 = 청크 하나

_id = chunk_id	doc_id	text_hash	text	text_tokens	embedding	메타데이터
같은 청크는 덮어씀	지울 때 쓰는 원천 문서 키	청크 재사용 판단 용도	보여 주기용, 색인 안 함	토큰화된 낱말	text_tokens를 벡터화	카드명, 회원 키 등 필터

doc_id 만드는 법

D1_27223010abe31291	_112f1e4abea05436	_0000
문서 키	본문 해시	청크 순번

→ **청크 순번의 앞 칸만 남기면 doc_id.**
본문이 바뀌어도 문서 키는 그대로

무단전재 및 배포 금지

③ 검색 엔진에 쓰기 코드

문서별로 묶고, 옛 청크를 지우고, 새 청크를 한꺼번에 넣은 뒤 바로 보이게 함

search_branch.py 발췌 (타입 표기와 설명 주석은 줄임)

```
def step3_write_documents(client, index, documents):
    # ① 문서(doc_id)별로 묶음
    grouped = {}
    for document in documents:
        grouped.setdefault(document["doc_id"], []).append(document)
    # ② 문서마다 옛 청크를 통째로 지움
    deleted = 0
    for doc_id in grouped:
        response = client.delete_by_query(
            index=index,
            body={"query": {"term": {"doc_id": doc_id}}},
            refresh=True, conflicts="proceed")
        deleted += response.get("deleted", 0)
    # ③ 새 청크를 한꺼번에 넣음 (_id = chunk_id)
    actions = [{"_index": index, "_id": d["chunk_id"], "_source": d}
               for docs in grouped.values() for d in docs]
    helpers.bulk(client, actions, max_retries=3)
    # ④ 바로 검색되게 반영
    client.indices.refresh(index=index)
    return WriteReport(deleted, len(actions), tuple(grouped))
```

1 문서별로 묶음
여러 문서가 한꺼번에 들어와도 문서마다 따로 지움

2 옛 청크 지우기
term doc_id로 그 문서 청크를 모두 지움. refresh=True라 바로 반영되고, conflicts="proceed"는 충돌이 나도 멈추지 않음

3 한꺼번에 넣기
_id를 chunk_id로 고정해 같은 입력을 두 번 넣어도 건수가 그대로임. 일부가 실패하면 BulkIndexError

4 반영
넣은 청크가 다음 검색부터 보임

시험 결과: 옛 청크 9개 지움 → 10개 넣음, 고아 0개

운영 중인 인덱스에 바로 쓰면 ②와 ③ 사이에 그 문서가 잠깐 안 보임전체 재색인은 새 세대에 쓰고 ④단계 별칭 전환으로 바꿈

무단전재 및 배포 금지